

Justificación de operaciones de la práctica de PRO2

Primavera 2024

COMERCIAR

Inicializaciones. Inicialmente, no hemos comprobado ningún producto de ninguna ciudad de las dos, por lo tanto, inicializamos it a inventario.begin() e it2 a c.inventario.begin(), satisfaciendo la primera y la segunda parte de la invariante, dado que no puede existir ningún iterador que apunte al final de un inventario (.end()) dado que la precondition se cumple.

Condición de salida. Como que la postcondición dice que se han intercambiado los productos que le sobran a una ciudad y necesitaba la otra, tendremos este resultado (según la invariante) cuando el último elemento de unos de los inventarios sea el anterior al que marca it o it2, y esto pasará cuando it o it2 (uno de los dos) sea igual a inventario.end() o c.inventario.end() respectivamente, el elemento ficticio del final del inventario. Por lo tanto, nos quedamos mientras it != inventario.end() e it2 != c.inventario.end().

Cuerpo del bucle. Supondremos que al inicio de una iteración cualquiera del bucle se cumplen la invariante y la condición de entrada en el bucle. Veremos que después de ejecutar las instrucciones del bucle se vuelve a hacer cierta la invariante. En primer lugar, sabemos que it e it2 no dan error, puesto que por la invariante tenemos que apuntan a un elemento de inventario o es inventario.end() o c.inventario.end() y si entramos al bucle no puede ser este último caso.

- En el caso en el cual las dos ciudades hayan podido intercambiar productos (o bien tienen un mismo producto, pero no han podido realizar ningún intercambio), ya podemos incrementar it e it2 (a la vez) asegurando que satisfacemos la invariante.
- En el supuesto caso de que no tengan un mismo producto, ya podríamos incrementar it o it2 (solo uno de los dos) asegurando también que satisfacemos de nuevo la invariante.

Finalización (función de cota). A cada vuelta decrece la distancia entre *inventario.end()* e *it* y entre *c.inventario.end()* y *it2*. Por tanto, las funciones de cota son la distancia entre *it* e *inventario.end()* y entre *it2* y *c.inventario.end()*.

Invariante: *it* apunta a un elemento de *inventario* o es *inventario.end()* e *it2* apunta a un elemento de *c.inventario* o es *c.inventario.end()*;

Se han comerciado cada elemento de *inventario* desde el inicio hasta la posición anterior a la marca de *it*. También se han comerciado cada elemento desde el inicio hasta la posición anterior de *it2*. A partir de *it* hasta el final de *inventario*, los elementos son los mismos que previamente antes de entrar al bucle. A partir de *it2* hasta el final de *c.inventario*, los elementos son los mismos previamente antes de entrar al bucle.

El programa anteriormente descrito queda como:

```

86 // Pre: Las dos ciudades existen, son diferentes y tienes por lo menos un
87 // producto
88 // Post: Se han intercambiado los productos que le sobran a una ciudad y
89 // necesitaba la otra
90 void Ciudad::comerciar_inventario(Ciudad &c,
91 const map<int, pair<int, int>> &catalogo)
92 {
93     auto it = inventario.begin();
94     auto it2 = c.inventario.begin();
95     while (it != inventario.end() and it2 != c.inventario.end()) {
96         // Si el identificador del producto de la primera ciudad es mas
97         // pequeño, significa que la segunda ciudad no tiene ese producto, así
98         // que tenemos que pasar al siguiente producto de la primera ciudad
99         // para comprobar si coincide con el de la segunda
100         if (it->first < it2->first) ++it;
101         // Si el identificador del producto de la segunda ciudad es mas
102         // pequeño, significa que la primera ciudad no tiene ese producto, así
103         // que tenemos que pasar al siguiente producto de la segunda ciudad
104         // para comprobar si coincide con el de la primera
105         else if (it->first > it2->first) ++it2;
106         // En el caso que los dos identificadores sean iguales, hemos
107         // encontrado un producto en común de las dos ciudades
108         else {
109             auto it3 = catalogo.find(it->first);
110             int peso1 = it3->second.first;
111             int vol1 = it3->second.second;
112
113             auto it4 = catalogo.find(it2->first);
114             int peso2 = it4->second.first;
115             int vol2 = it4->second.second;
116
117             // La primera ciudad quiere comprar (tiene1 < quiere1) y la segunda
118             // quiere vender (tiene2 > quiere2)
119             if (it->second.first < it->second.second and
120                 it2->second.first > it2->second.second) {
121                 // La cantidad intercambiada será la mínima que
122                 // quieran comprar/vender las ciudades
123                 int intercambio = min(it->second.second - it->second.first,
124                                     it2->second.first - it2->second.second);
125                 // Cambios que sufre la primera ciudad
126                 it->second.first += intercambio;
127                 peso_total += (peso1 * intercambio);
128                 volumen_total += (vol1 * intercambio);
129                 // Cambios que sufre la segunda ciudad
130                 it2->second.first -= intercambio;
131                 c.peso_total -= (peso2 * intercambio);
132                 c.volumen_total -= (vol2 * intercambio);
133             }
134             // La segunda ciudad quiere comprar (tiene2 < quiere2) y la primera
135             // quiere vender (tiene1 > quiere1)
136             else if (it2->second.first < it2->second.second and
137                 it->second.first > it->second.second) {
138                 // La cantidad intercambiada será la mínima que
139                 // quieran comprar/vender las ciudades
140                 int intercambio = min(it2->second.second - it2->second.first,
141                                     it->second.first - it->second.second);
142                 // Cambios que sufre la segunda ciudad
143                 it2->second.first += intercambio;
144                 c.peso_total += (peso2 * intercambio);
145                 c.volumen_total += (vol2 * intercambio);
146                 // Cambios que sufre la primera ciudad
147                 it->second.first -= intercambio;
148                 peso_total -= (peso1 * intercambio);
149                 volumen_total -= (vol1 * intercambio);
150             }
151             // Como ya hemos visto el producto que tienen en común las dos
152             // ciudades, pasamos al siguiente
153             ++it;
154             ++it2;
155         }
156     }
157 }

```

HACER VIAJE (FUNCIÓN QUE CALCULA EL CAMINO)

Caso sencillo 1. Si el árbol es vacío, esto significa que este tiene cero ciudades y entonces no existe ningún camino más provechoso que otro para el barco, porque no hay ningún camino posible a seguir.

Caso sencillo 2. Si $\underline{nidc} == 0$ y $\underline{nidv} == 0$, esto significa que el barco se ha quedado sin productos y entonces debemos parar la ejecución, porque no puede ofrecer más productos a las demás ciudades restantes.

Caso recursivo 1. Si el árbol no es vacío y el barco aún tiene productos, los enteros $\underline{nidc}$ y $\underline{nidv}$ son naturales tales que $|\underline{nidc} - \underline{nidv}| > 0$. Por tanto, la precondition se evalúa *true* y la llamada recursiva al subárbol izquierdo es correcta.

Caso recursivo 2. Si el árbol no es vacío y el barco aún tiene productos, los enteros $\underline{nidc}$ y $\underline{nidv}$ son naturales tales que $|\underline{nidc} - \underline{nidv}| > 0$. Por tanto, la precondition se evalúa *true* y la llamada recursiva al subárbol derecho es correcta.

Decrecimiento (función de cota). A cada llamada recursiva el árbol $\underline{t}$ va reduciendo el número de nodos y los valores $\underline{nidc}$ y $\underline{nidv}$ son naturales que se hacen cada vez más pequeños. Por tanto, la función de cota es la altura del árbol $\underline{t}$ y los valores $\underline{nidc}$ y $\underline{nidv}$.

El programa anteriormente descrito queda como:

```

263 // Pre: Cierta
264 // Post: El resultado es el camino más conveniente posible
265 list<string> Rio::camino(const BinTree<string> &t,
266 const map<string, Ciudad> &info, int &nidc, int &nidv, int idc, int idv)
267 {
268     list<string> ret;
269     // Caso base
270     if (t.empty()) ret = list<string>();
271     else {
272         // Caso general
273         int comprados = nidc;
274         int vendidos = nidv;
275
276         /* EVALUACION DE PRODUCTOS */
277         auto it = info.find(t.value());
278         if (it->second.existe(idc)) {
279             int tiene1 = it->second.consultar_tiene(idc);
280             int quiere1 = it->second.consultar_quiere(idc);
281             // El barco compra productos
282             if (tiene1 > quiere1 and nidc != 0) {
283                 int compra = min(tiene1 - quiere1, nidc);
284                 nidc -= compra;
285             }
286         }
287         if (it->second.existe(idv)) {
288             int tiene2 = it->second.consultar_tiene(idv);
289             int quiere2 = it->second.consultar_quiere(idv);
290             // El barco vende productos
291             if (tiene2 < quiere2 and nidv != 0) {
292                 int venta = min(quiere2 - tiene2, nidv);
293                 nidv -= venta;
294             }
295         }
296
297         // Caso 1: El barco ya no tiene mas productos (paramos la ejecucion)
298         if (nidc == 0 and nidv == 0) {
299             ret = list<string>(1, t.value());
300         }
301         else {
302             /* LLAMADAS RECURSIVAS */
303             int comp_izq = nidc;
304             int vend_izq = nidv;
305             list<string> izq = camino(t.left(), info, comp_izq, vend_izq, idc,
306 idv);
307             int comp_der = nidc;
308             int vend_der = nidv;
309             list<string> der = camino(t.right(), info, comp_der, vend_der, idc,
310 idv);
311             /* HI: izq es el camino mas provechoso del subarbol izquierdo y
312 der el camino mas provechoso del subarbol derecho
313 */
314
315             // Caso 2: El nodo es una hoja y no se ha comprado y vendido ningun
316 // producto
317             if (comp_izq == nidc and vend_izq == nidv and comp_der == nidc
318 and vend_der == nidv) {
319                 if (comprados == nidc and vendidos == nidv)
320                     ret = list<string>();
321
322                 else ret = list<string>(1, t.value());
323             }
324             else {
325                 // Caso 3: Ruta izquierda
326                 if ((comp_izq + vend_izq) < (comp_der + vend_der)) {
327                     izq.push_front(t.value());
328                     nidc = comp_izq;
329                     nidv = vend_izq;
330                     ret = izq;
331                 }
332                 // Caso 4: Ruta derecha
333                 else if (comp_izq + vend_izq > comp_der + vend_der) {
334                     der.push_front(t.value());
335                     nidc = comp_der;
336                     nidv = vend_der;
337                     ret = der;
338                 }
339                 // Caso 5: Depende
340                 else {
341                     if (izq.size() <= der.size()) {
342                         izq.push_front(t.value());
343                         nidc = comp_izq;
344                         nidv = vend_izq;
345                         ret = izq;
346                     }
347                     else {
348                         der.push_front(t.value());
349                         nidc = comp_der;
350                         nidv = vend_der;
351                         ret = der;
352                     }
353                 }
354             }
355         }
356     }
357     return ret;
358 }

```